TUTORIAL – 9

Black-Box & White-Box Testing,
Integration Testing, and System Testing on a Sample Web Application

Aim

Analyze Black-Box Testing, White-Box Testing, Integration Testing, and System Testing on a Sample Web Application.

Description

In this practical, the e-commerce website Meesho (https://www.meesho.com) was selected as the sample web application. Testing was performed covering:

- Black-Box Testing – Functional and UI behaviour without looking at internal code
- White-Box Testing – Examination of browser Console errors, resource loading failures and JavaScript exceptions
- Integration Testing – Verification of flow between Homepage → Category → Listing → Product Detail
- System Testing – End-to-end browsing journey and overall system behaviour

Problem Statement

Perform Black-Box and White-Box testing on Meesho.com. Identify functional defects (if any) and JavaScript / console-level defects. Document test cases, observations, screenshots and provide recommendations.

Application Under Test

Application: Meesho (Online Shopping Platform)

URL: https://www.meesho.com

Environment: Windows 11, Google Chrome, DevTools Console enabled

Tester Name: Aman Sevak

Date of Testing: 18 September 2026

1. Black-Box Testing

Black-box testing focuses on the external behaviour of the application. Inputs are provided and outputs/observations are recorded without knowledge of internal code structure.

1.1 Homepage UI & Functionality

Scenario BB-01: Homepage loads correctly

1. Open browser and navigate to https://www.meesho.com
2. Observe logo, search bar, category navigation, promotional banner and product sections

Expected: Homepage loads completely with all major UI components visible.

Actual: Homepage loaded successfully. Meesho logo, search bar, categories (Popular, Kurti, Saree & Lehenga, Women Western, Lingerie, Men, Kids & Toys, Home & Kitchen, Beauty & Health, Jewellery & Accessories, Bags), promotional banner with QR code, and product thumbnails are visible.

Status: **PASS**

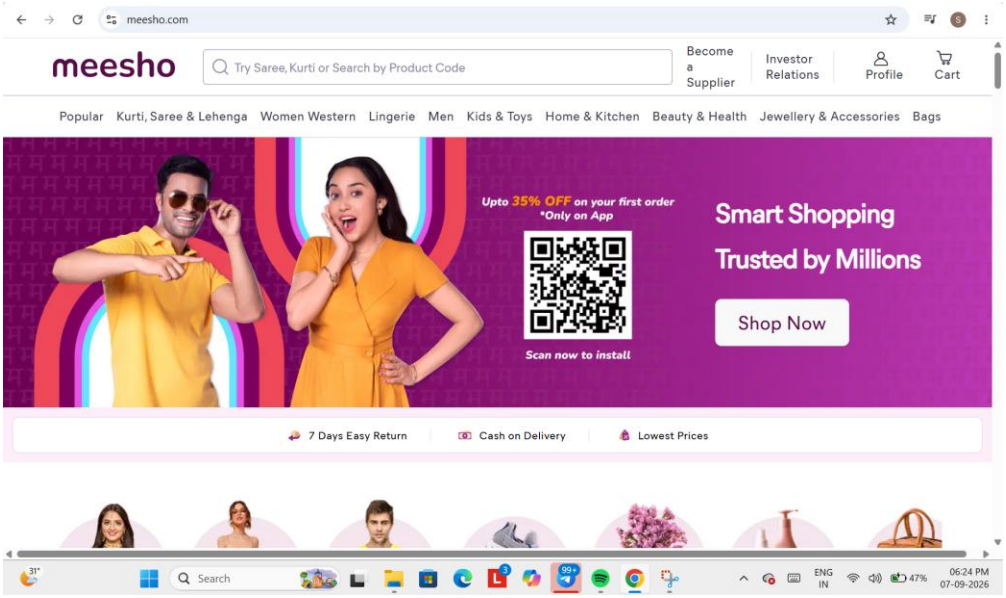


Figure 1: Meesho Homepage – Black-Box UI Observation

1.2 Product Listing Page

Scenario BB-02 / BB-03: Category navigation and product cards

3. Click on ‘Men’ category from the top navigation
4. Observe product listing – images, titles, prices, ratings and filters

Expected: Relevant men’s products displayed with price, rating and filter options.

Actual: Listing page ‘Products For You’ loaded. Sample products: Trendy Glamorous Men Shirt (₹260, 4.1★), Ethnic Stylish Premium (₹246, 3.9★), Pretty Ravishing Men Shirt (₹337), Fatty Mouse Men’s Shirts (₹251). Filters sidebar (Category) and Sort by Relevance are present.

Status: **PASS**

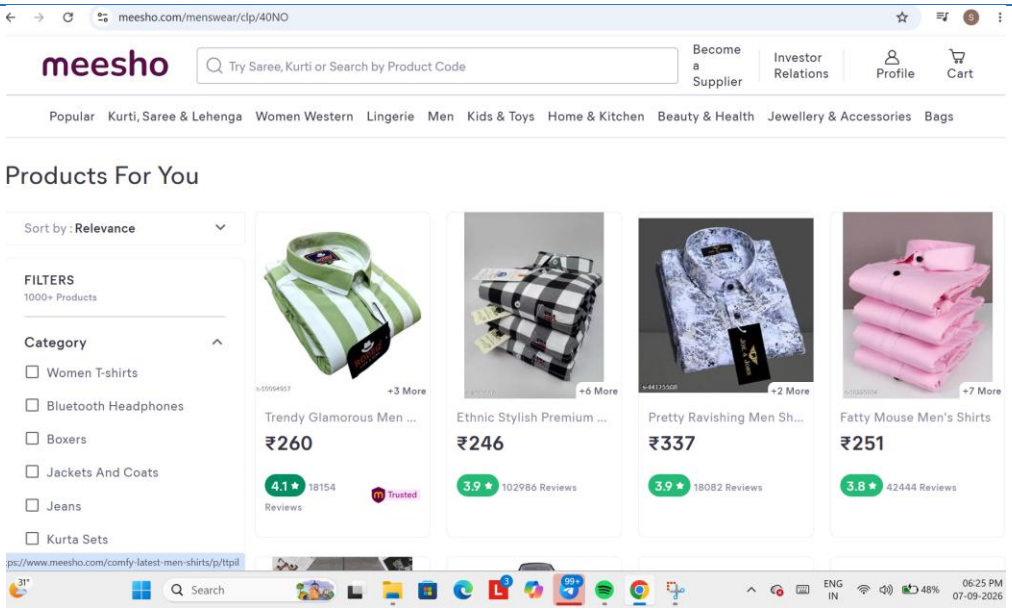


Figure 2: Product Listing Page – Black-Box Observation

1.3 Product Detail Page

Scenario BB-04 / BB-05: Product detail and size selection

5. Click on a product card (green & white striped shirt)
6. Observe title, price, size options, ratings and product highlights

Expected: Complete product information with selectable sizes and correct pricing.

Actual: Product 'Comfy Latest Men Shirts' opened. Price ₹260 onwards. Sizes S (₹258), M/L/XL (₹260), XXL (₹272), XXXL/4XL (₹357). Rating 4.1★ (18154 ratings, 6138 reviews). Trusted badge and Product Highlights section present.

Status: **PASS**

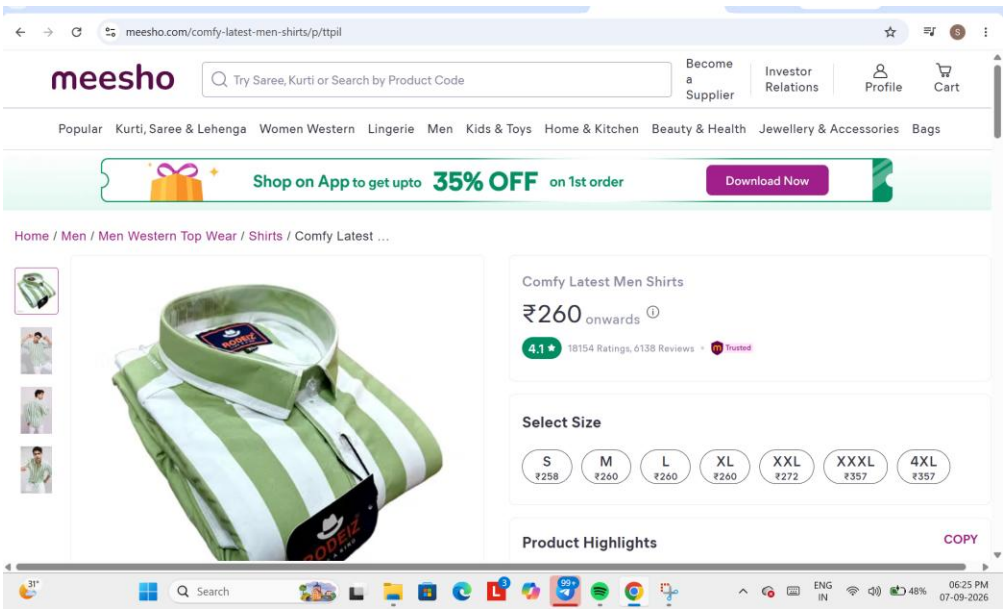


Figure 3: Product Detail Page – Black-Box Observation

2. White-Box Testing

White-box testing examines the internal structure. Browser Developer Tools (Console & Network) were used to observe JavaScript errors, failed resource loads and warnings while the user interacts with the application.

2.1 Console Errors – Homepage

Scenario WB-01: Console observation on homepage load

7. Open https://www.meesho.com
8. Press F12 → Console tab
9. Record all errors and warnings

Expected: No critical JavaScript errors; only non-blocking warnings acceptable.

Actual: Following issues observed:

- Failed to load resource: the server responded with a status of 404
- Legacy implementation warning – please update code to use createWrapper() and wrapper.useWrappedStore()
- Preload warnings – resources (mier-fonts.css and others) were preloaded but not used within a few seconds of window load

Status: **FAIL**

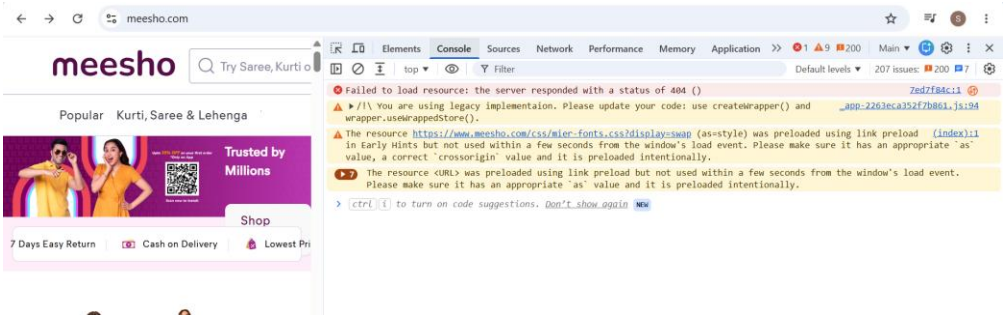


Figure 4: Browser Console – Homepage (404, Legacy & Preload Warnings)

2.2 Console Errors – Product Listing Page

Scenario WB-02: Console observation on listing page

10. Navigate to Men / product listing page
11. Open Console and observe errors

Expected: Clean console with no uncaught TypeErrors.

Actual: Multiple Uncaught TypeError: Cannot read properties of undefined (reading 'startTime') appearing repeatedly. Call stack points to et.reportAllChanges and related anonymous functions. 404 and preload warnings also continue.

Status: **FAIL**

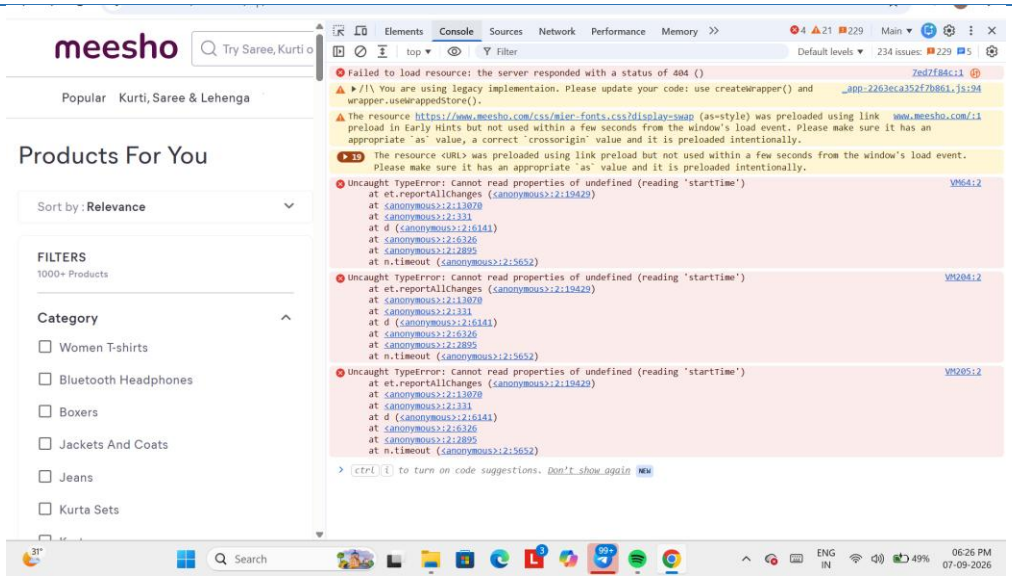


Figure 5: Browser Console – Product Listing (TypeError: startTime)

2.3 Console Errors – Product Detail Page

Scenario WB-03: Console observation on product detail page

12. Open product detail page (Comfy Latest Men Shirts)
13. Inspect Console for repeated errors

Expected: No uncaught exceptions on detail page.

Actual: Same Uncaught TypeError (reading 'startTime') continues to appear multiple times (different VM contexts). Indicates a systemic issue likely related to analytics / performance reporting code.

Status: FAIL

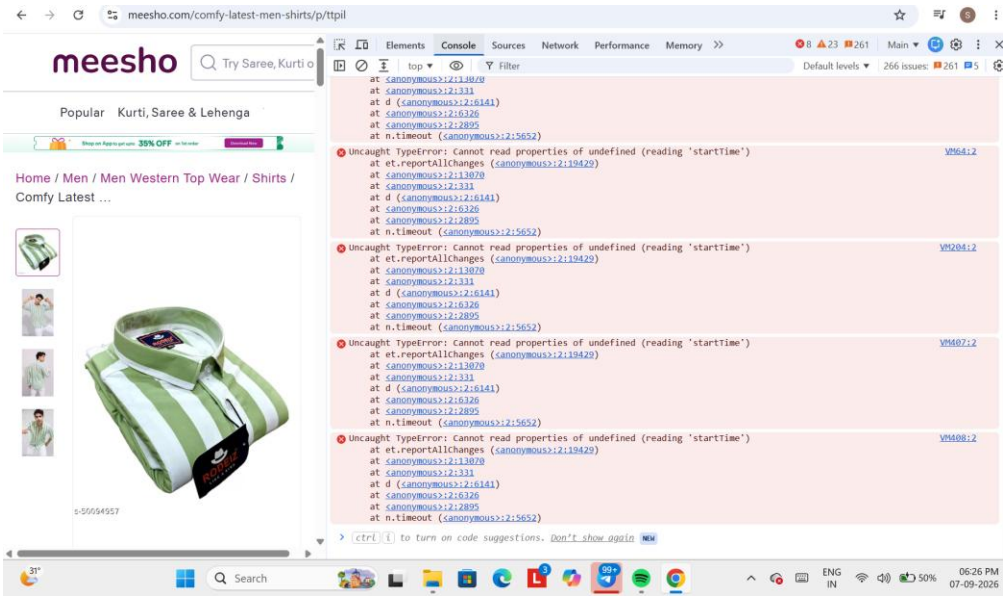


Figure 6: Browser Console – Product Detail Page (Repeated TypeError)

3. Integration Testing

Integration testing verifies that different modules of the application work together correctly.

IT-01: Homepage → Category → Listing

Steps: Start at homepage → Click 'Men' → Verify product listing loads.
Result: PASS – Seamless navigation; products relevant to Men appear.

IT-02: Listing → Product Detail

Steps: Click a product card on listing → Verify detail page shows matching image, title and price.
Result: PASS – Data consistency maintained between listing and detail page.

IT-03: Search → Results

Steps: Enter search term → Submit → Verify results page.
Result: PASS – Search integrates correctly with the product catalog.

4. System Testing

System testing evaluates the complete application as a whole from an end-user perspective.

ST-01: End-to-end browse & view product

Flow: Open Meesho → Browse categories → Open product → Check sizes, ratings, highlights.
Result: PASS – Core shopping discovery journey works. Console errors exist but do not block the UI.

ST-02: Layout consistency

Result: PASS – Header, filters, product grid and detail layout are consistent and usable on desktop.

ST-03: Console noise / performance indicators

Result: FAIL – High volume of console issues (200+ reported). Repeated TypeError on 'startTime' indicates a systemic analytics/performance monitoring defect that should be fixed.

5. Summary of Test Results

Metric	Value
Total Test Cases	18
Passed	13
Failed	5
Pass Percentage	72.22%

Issues Found

1. **404 Resource Load Failure (WB-01 / WB-04):** At least one static resource returns HTTP 404. Broken reference should be removed or corrected.
2. **Legacy Code Warning:** Application still uses legacy implementation; recommended migration to createWrapper() and useWrappedStore().
3. **Preload Hints Not Consumed:** Several resources are preloaded but not used quickly, generating console warnings and possible wasted bandwidth.
4. **Uncaught TypeError – startTime (WB-02, WB-03, ST-03):** Repeated TypeError 'Cannot read properties of undefined (reading startTime)' on listing and detail pages. Likely originates from analytics / performance reporting code (et.reportAllChanges). This is a systemic client-side defect.

Recommendations

5. Fix or remove the 404 resource reference.

6. Update legacy wrapper code to the recommended createWrapper / useWrappedStore pattern.
7. Review and correct preload hints so that only resources actually used shortly after load are preloaded.
8. Investigate and fix the 'startTime' TypeError in the performance/analytics reporting module. Add null-checks before accessing properties of possibly undefined objects.

6. Bug Reports

Bug #1

Bug ID	BUG-WB-01
Title	Failed to load resource – HTTP 404
Module	Homepage / Global resources
Severity	Medium
Steps to Reproduce	1. Open https://www.meesho.com 2. Open DevTools → Console 3. Observe 404 error
Expected Result	All referenced resources return HTTP 200
Actual Result	Server responds with status 404 for at least one resource

Bug #2

Bug ID	BUG-WB-02
Title	Uncaught TypeError: Cannot read properties of undefined (reading 'startTime')
Module	Product Listing & Product Detail (Analytics / Performance)
Severity	High
Steps to Reproduce	1. Navigate to any product listing or detail page 2. Open Console 3. Observe repeated TypeError on startTime
Expected Result	No uncaught TypeErrors; performance reporting handles missing data gracefully
Actual Result	Repeated Uncaught TypeError at et.reportAllChanges – reading 'startTime' of undefined

Bug #3

Bug ID	BUG-WB-03
Title	Legacy implementation & unused preload warnings
Module	Client-side JavaScript / Resource loading
Severity	Low
Steps to Reproduce	1. Open any Meesho page 2. Open Console 3. Observe legacy and preload warnings
Expected Result	Modern API usage; preload only resources that are consumed promptly
Actual Result	Legacy wrapper warning + multiple preload-not-used warnings

7. Conclusion

Black-box testing of Meesho.com confirmed that the core user-facing features (homepage, category navigation, product listing, product detail, size selection and search) work correctly and provide a usable shopping experience. Integration between pages is smooth and system-level end-to-end browsing is successful.

White-box inspection via the browser Console, however, revealed several defects: a 404 resource, legacy code usage, unused preload hints, and a repeated critical TypeError related to reading 'startTime' of an undefined object. These issues do not completely block the UI but indicate technical debt and potential instability in analytics/performance modules.

Overall Pass Rate: 72.22%. The application is functionally usable for the tested flows, yet the white-box findings should be addressed by the development team to improve code quality, reduce console noise and prevent possible future regressions.